



UNIVERSITY OF AMSTERDAM
Amsterdam Business School

Data Science in Online Marketing

Online Advertising - CTR Prediction

Maarten Soomer



Course Outline

Date	Topic	Assignment
3 april	Recommenders: Content Based	
10 april	Recommenders: Collaborative Filtering	Yes
17 april	Recommenders: Guest Lecture Bol.com Online advertising: Introduction	
24 april	Online advertising: Reinforcement Learning	
1 may	Online advertising: CTR prediction	Yes
15 may	Online advertising: Media Mix Modeling & Conversion Attribution	
22 may	Revision	
29 may	Digital exam on location	

Measuring Performance in online marketing

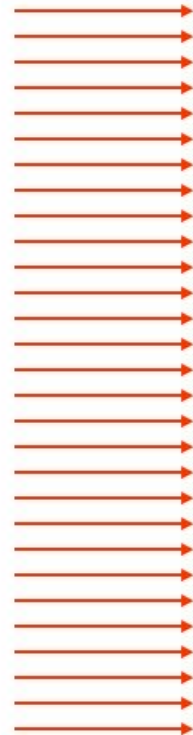
Click-through rate (CTR) =

$$\frac{\# \text{ click throughs}}{\# \text{ impressions}}$$

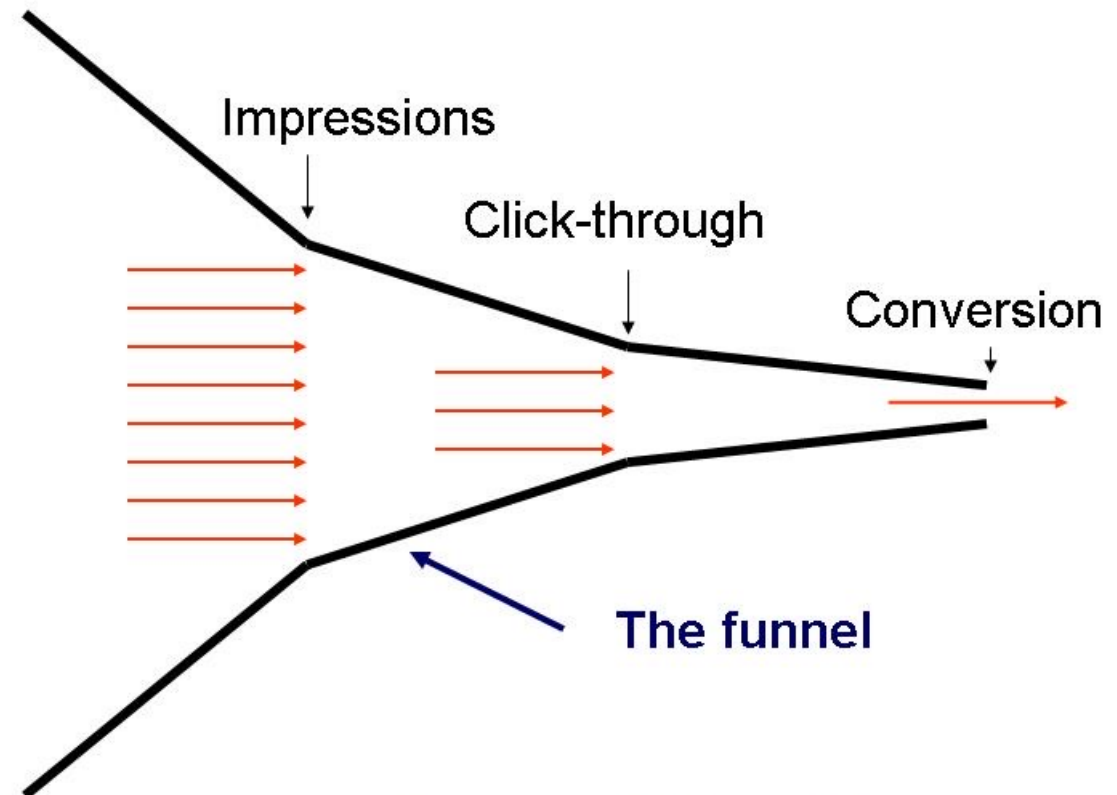
Conversion rate (CR) =

$$\frac{\text{conversions}}{\# \text{ click throughs}}$$

Target audience

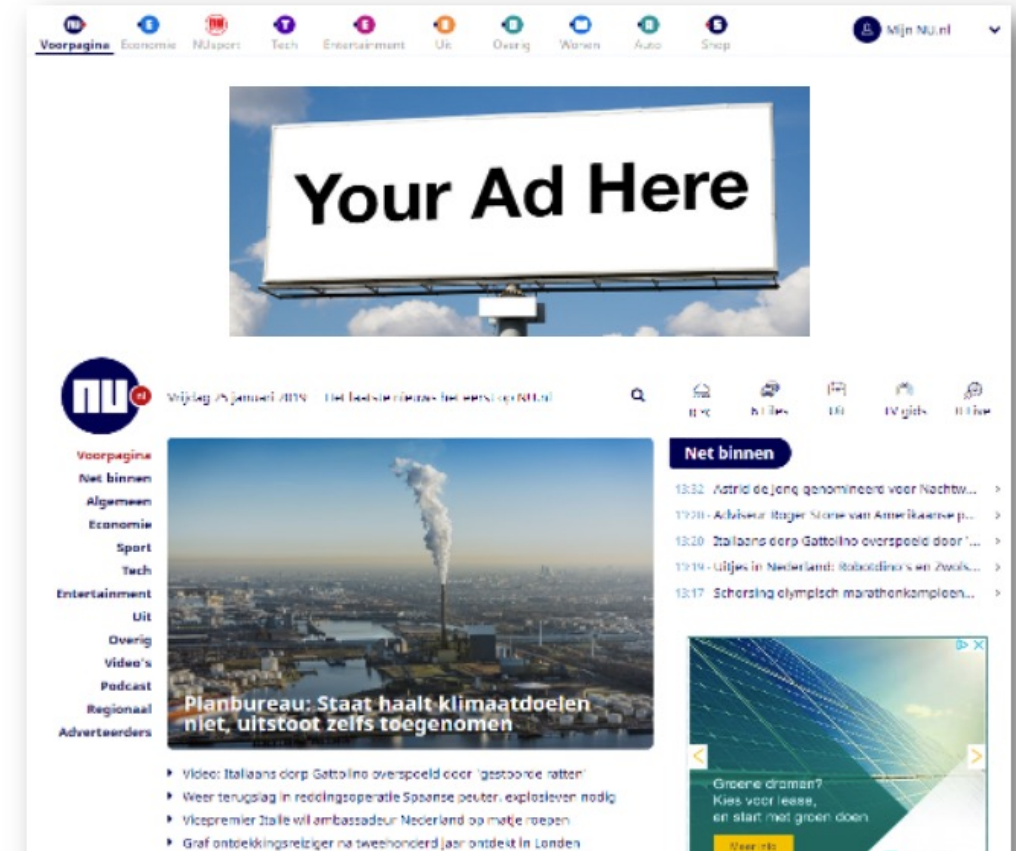
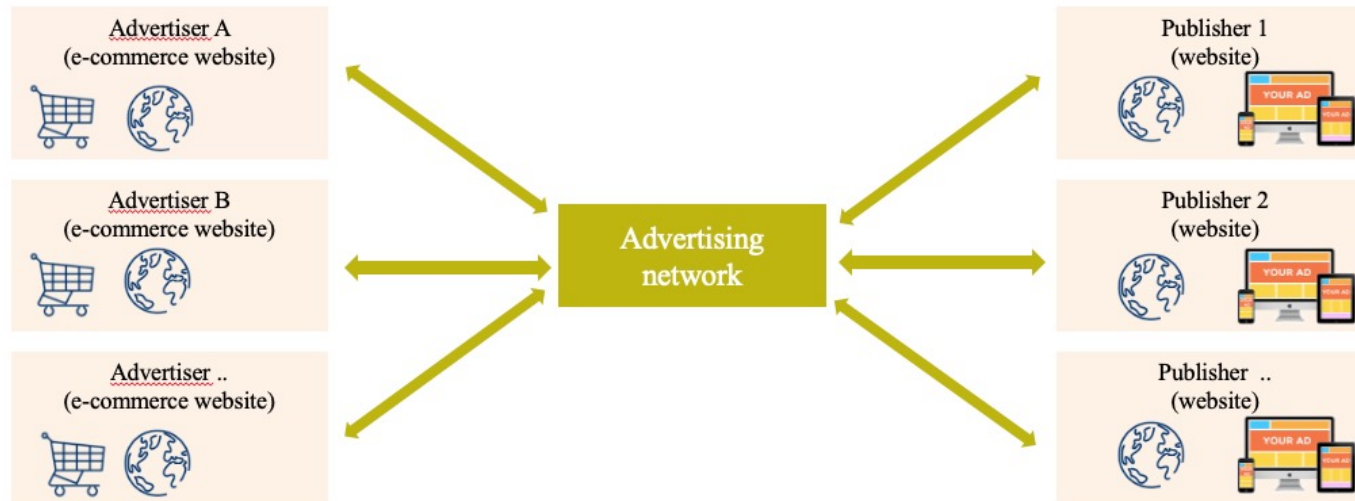


The marketing funnel





Advertising Networks





CTR Prediction

Last week: Overall CTR estimation for a few versions of an ad

- How **Bayesian Updating** can improve CTR estimates in the start-up phase
- How **Multi-Armed Bandits** can balance exploration vs exploitation

Today: CTR prediction with many features (context)

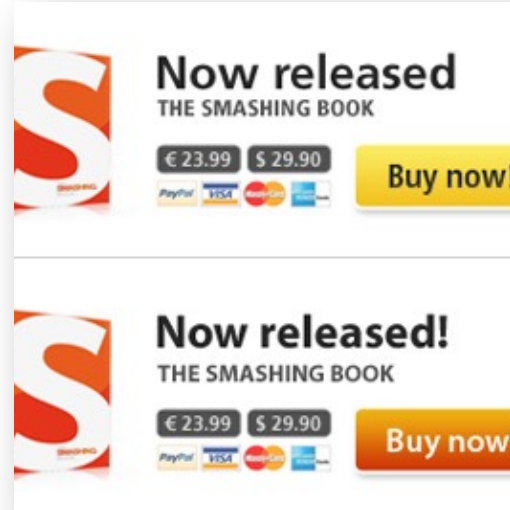
- Problem characteristics, algorithm selection and model evaluation
- Algorithms
 - **Naïve Bayes**
 - **Logistic Regression**

CTR Prediction - Features



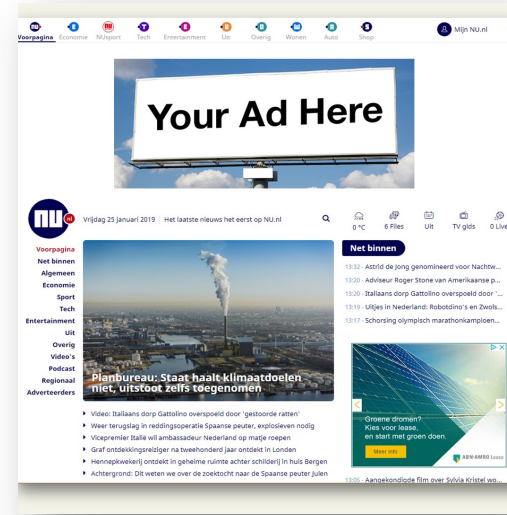
Visitor

- Device (phone, tablet, pc)
- Operating System
- Location
- Browsing behaviour
- ...



Advertisement

- Advertiser
- Product Category
- Ad version
- Ad text
- ...



Publisher

- Website/App
- Ad Position
- Page contents
- ...



Context

- Time of day
- Day of week
- Month/season
- Weather
- ...

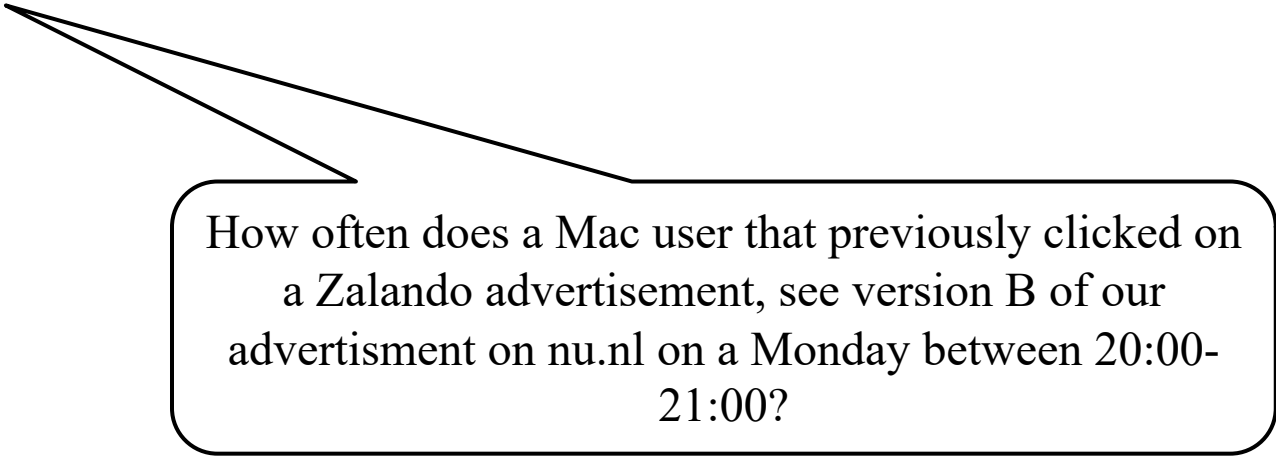


CTR Prediction - Features

- Variables providing information about an advertisement impression
- Most of these features are categorical, e.g., device, operating system
 - Some can have many different values, e.g., website
- This typically leads to many possible combinations of feature values, each having very limited or even no observations

Example:

- Device = {phone, tablet, pc}
- Weekday = {Monday, Tuesday, ...}



How often does a Mac user that previously clicked on a Zalando advertisement, see version B of our advertisement on nu.nl on a Monday between 20:00-21:00?



CTR Prediction – Feature Interactions

It is important to consider feature interactions:

- Assume young people respond very well to ads about games
- But for other age groups ads about games perform like other type of ads
 - Introduce a cross feature: $x_{young} x_{game}$

There are many possible feature interactions, further enhancing the sparseness and size of the data.

Feature interactions can introduce multi-collinearity or violate independence conditions.

(Manual) Feature selection might be required.



UNIVERSITY OF AMSTERDAM
Amsterdam Business School

CTR prediction model



CTR Prediction – Problem Characteristics

- How to select an algorithm for CTR prediction?
- How to evaluate a CTR prediction model?



CTR Prediction – Problem Characteristics

How to select an algorithm for CTR prediction?

- Classification Model (Click/No-Click)
 - Class Imbalance: typically click rates are low.
- Many categorical features with high cardinality
- Dynamic environment
 - Large number of observations (impressions) in a short time period
 - New observations come in continuously
 - New ads, websites appear often



CTR Prediction Model

Observations: Impression

- Outcome: $Y = \{\text{Click, No Click}\}$
- Features: $X = (X_1, X_2, X_3, \dots, X_k)$, e.g.,
 - Device = {phone, tablet, pc}
 - Weekday = {Monday, Tuesday, ...}
 - ..

What are we interested in?

Predict if an impression with given feature values leads to a click or not?

```
from sklearn.module import SomeClassifier

model = SomeClassifier()

# Train the model on a training dataset
model.fit(X_train, y_train)

# Predict class labels (Click or No-Click) for observations in X
model.predict(X)
```




CTR Prediction: Goal

Typical click rates are very low overall, so the predicted class will almost always be a “No Click”.

We are interested in impressions where the click probability is **relatively** high (compared to other impressions), e.g.,

	Actual click rate
phone	3%
pc	1%

In this case we would rather advertise on phones than on pc's

What are we interested in?

➔ We are interested in estimating the **probability of a click** for a given set of feature values:

$$P(Y = \text{Click} \mid X_1 = x_1, X_2 = x_2, \dots, X_k = x_k)$$

```
from sklearn.module import SomeClassifier

model = SomeClassifier()

# Train the model on a training dataset
model.fit(X_train, y_train)

# Predict probability of each class for observations in X
model.predict_proba(X)
```



CTR Prediction - Error Metric

What error metric do we choose for CTR prediction?

- Classification accuracy?



CTR Prediction - Error Metric

What error metric do we choose for CTR prediction?

- Classification accuracy?

Example

- Suppose we have 10.000 observations with 100 clicks and 990 no-clicks
- Suppose we have a model that always predicts a no-click

Observed	Prediction	# Observations
No Click	No Click	9.900
Click	No Click	100
Total		10.000

Accuracy?



CTR Prediction - Error Metric

What error metric do we choose for CTR prediction?

- ~~Classification accuracy~~

Example

- Suppose we have 10.000 observations with 100 clicks and 990 no-clicks
- Suppose we have a model that always predicts a no-click

Observed	Prediction	# Observations
No Click	No Click	9.900
Click	No Click	100
Total		10.000

Accuracy = 99%

A model with high accuracy could still classify all observations as “No Clicks”, but this doesn’t help us.



Error Metric

What error metric do we choose for CTR prediction?

- Mean Squared Error (on predicted probabilities)



Error Metric

What error metric do we choose for CTR prediction?

- Mean Squared Error

Example

- Suppose we have 10.000 observations with 100 clicks and 990 no-clicks
- Suppose we have a model that always predicts a 0% click probability

Observed	Prediction	# Observations
No Click	0%	9.900
Click	0%	100
Total		10.000

$$\text{MSE} = \frac{1}{10000} (9900 \times 0^2 + 100 \times (1 - 0)^2) = 0,01$$



Error Metric

What error metric do we choose for CTR prediction?

- ~~Mean Squared Error~~

We want a model that is able to (correctly) identify impressions where clicks are more likely, even if the data is very imbalanced. We really don't like clicks to have a predicted click probability of 0.

	Actual click rate
iPhone	3%
Android	1%

We would like a model that successfully predicts that the click rate for iPhone is 3 times as high as the click rate for Android.



Error Metric

What error metric do we choose for CTR prediction?

- **Log Loss**

$$= -[y \ln(p) + (1 - y) \ln(1 - p)]$$

with

- $y = 0$ (No Click) or 1 (Click)
- p is the predicted click probability

Error Metric

What error metric do we choose for CTR prediction?

- **Log Loss**

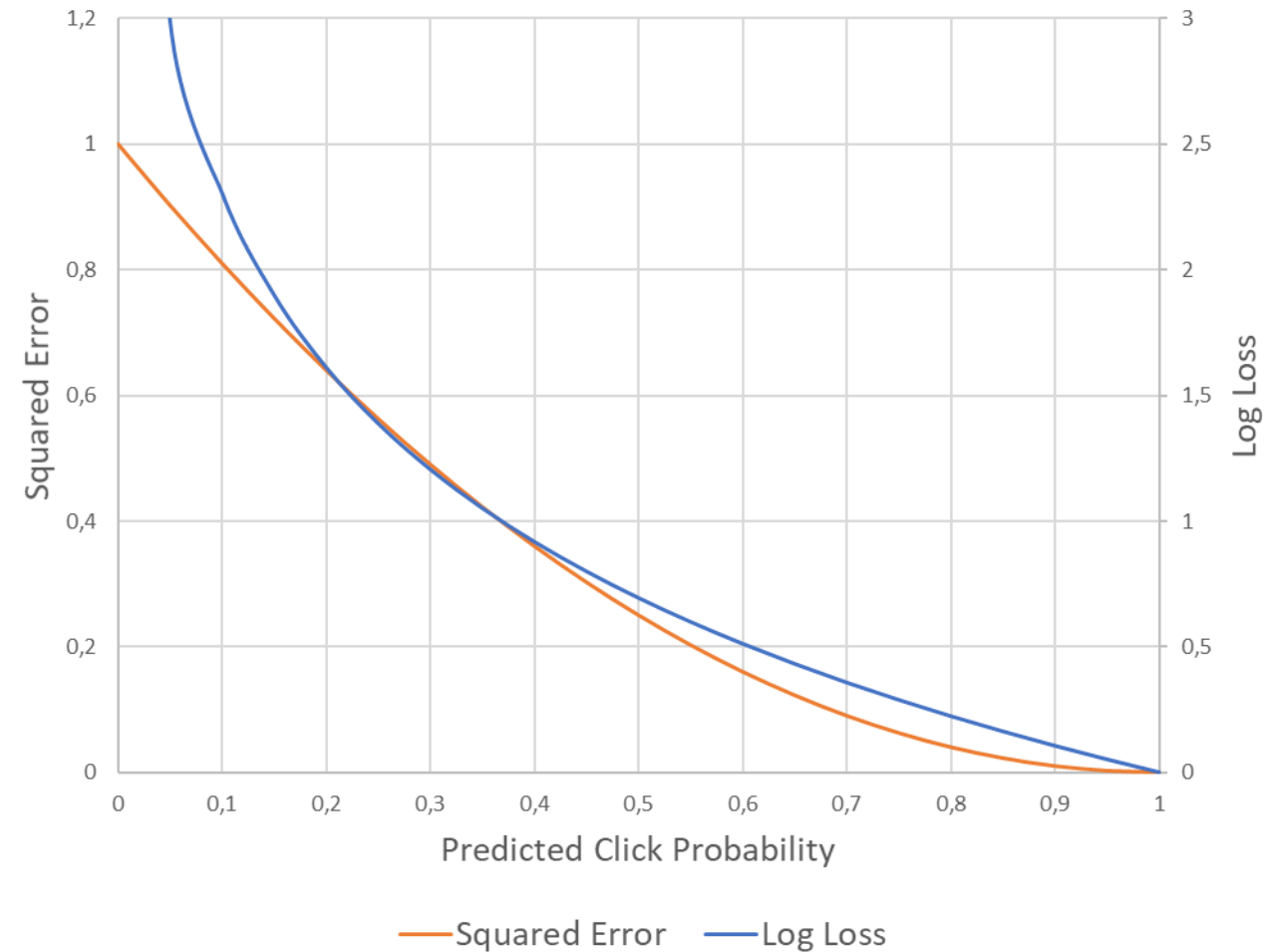
$$= -[y \ln(p) + (1 - y) \ln(1 - p)]$$

Click

with

- $y = 0$ (No Click) or 1 (Click)
- p is the predicted click probability

Prediction error for a Click





Error Metric

What error metric do we choose for CTR prediction?

- **Log Loss**

Example

- Suppose we have 10.000 observations with 100 clicks and 990 no-clicks
- Suppose we have a model that always predicts a 0% click probability

Observed	Prediction	# Observations	MSE	Log Loss
No Click	0%	9.900	0	0
Click	0%	100	1	∞
Total		10.000	0.01	∞



CTR prediction Goal and Error Metric

We are interested in estimating the **probability of a click** for a given set of feature values in order to find impressions where the click rate is *relatively* high.

Log Loss is an error metric on the predicted probabilities that heavily penalizes predictions of a zero probability for observed clicks (and the reverse). This error metric avoids selecting models that always predict a 0 probability because of imbalanced data (low overall click rates).

	Actual click rate
phone	3%
pc	1%



CTR Prediction - Online Learning

Dynamic Environment:

- Large number of observations (impressions) in a short time period
- New observations come in continuously
- New ads, websites appear often

Batch Learning

Train the model on the entire dataset at once

Algorithms: Linear Regression, Logistic Regression, Decision Tree, Random Forest, ...

Online Learning

Update the model observation by observation (as the data comes in)

Algorithms: Naive Bayes, Stochastic Gradient Descent, ...



CTR Prediction – Algorithm selection

How to select an algorithm for CTR prediction?

- Classification Model (Click/No-Click): predicting click probabilities
 - Handle class imbalance (low click rates)
 - Handle many categorical features with high cardinality
 - Online training to handle large number of streaming observations
 - Model Evaluation
 - Log Loss metric
-
- Naïve Bayes
 - Logistic Regression using Stochastic Gradient Descent

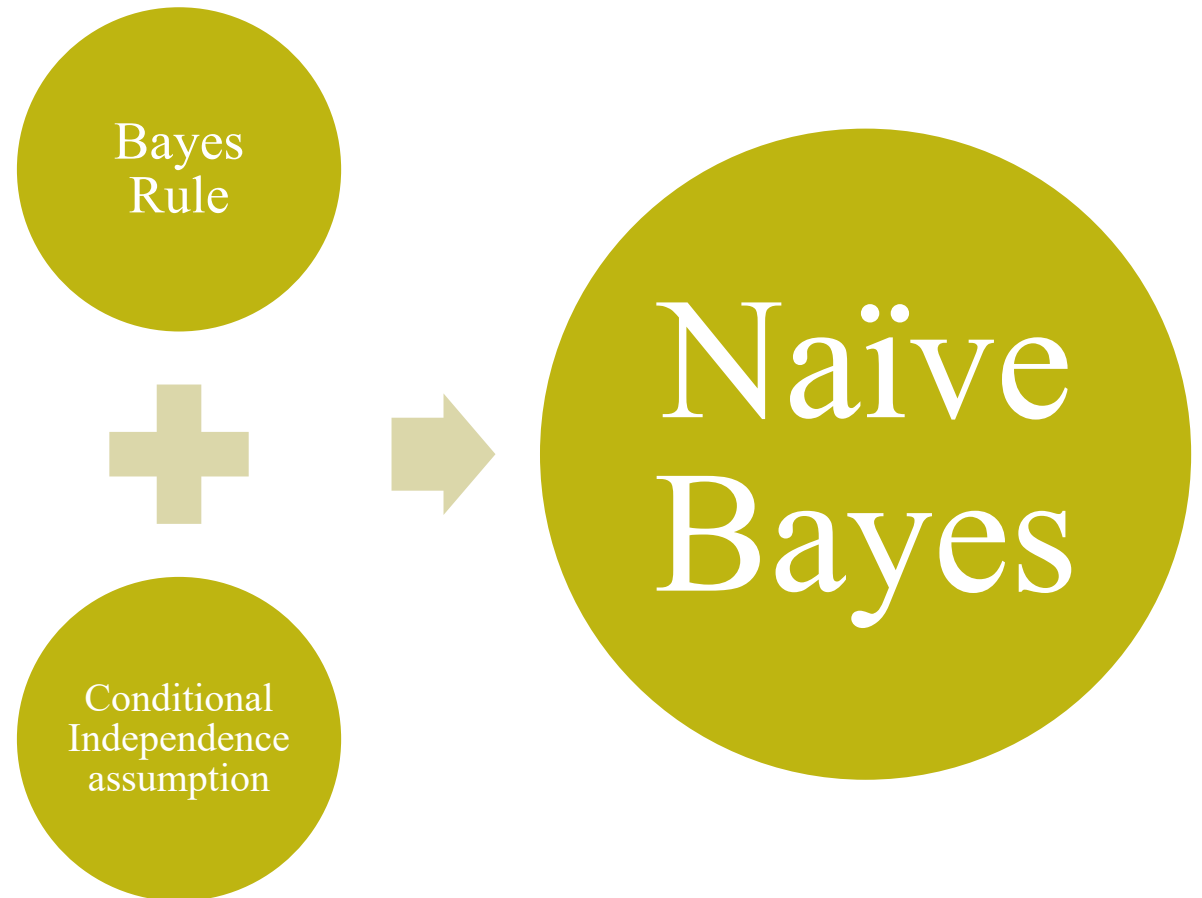


UNIVERSITY OF AMSTERDAM
Amsterdam Business School

Naïve Bayes

Naïve Bayes for CTR estimation

- Outcome: $Y = \{\text{Click, No Click}\}$
- Features: $X = (X_1, X_2, X_3, \dots, X_k)$, e.g.,
 - Device = {phone, tablet, pc}
 - Weekday = {Monday, Tuesday, ...}
 - ..
- We are interested in estimating
 $P(Y = \text{Click} \mid X_1 = x_1, X_2 = x_2, \dots, X_k = x_k)$





Naïve Bayes for CTR estimation

$$P(\text{Click} \mid X_1 = x_1, X_2 = x_2, \dots)$$

$$= P(X_1 = x_1, X_2 = x_2, \dots \mid \text{Click}) \frac{P(\text{Click})}{P(X_1 = x_1, X_2 = x_2, \dots)}$$

$$= \frac{P(X_1 = x_1, X_2 = x_2, \dots \mid \text{Click}) P(\text{Click})}{P(X_1 = x_1, X_2 = x_2, \dots \mid \text{Click}) P(\text{Click}) + P(X_1 = x_1, X_2 = x_2, \dots \mid \text{No Click}) P(\text{No Click})}$$

$$= \frac{[P(X_1 = x_1 \mid \text{Click}) * P(X_2 = x_2 \mid \text{Click}) * \dots] P(\text{Click})}{[P(X_1 = x_1 \mid \text{Click}) * P(X_2 = x_2 \mid \text{Click}) * \dots] P(\text{Click}) + [P(X_1 = x_1 \mid \text{No Click}) * P(X_2 = x_2 \mid \text{No Click}) * \dots] P(\text{No Click})}$$

$$= \frac{\prod_{i=1}^k P(X_i = x_i \mid \text{Click}) P(\text{Click})}{\prod_{i=1}^k P(X_i = x_i \mid \text{Click}) P(\text{Click}) + \prod_{i=1}^k P(X_i = x_i \mid \text{No Click}) P(\text{No Click})}$$

Bayes
Rule

Conditional
Independence
assumption



Conditional Independence assumption

Given the outcome of the trial the value of feature A is independent of the value of feature B

Example:

- Features:
 - Device = {phone, tablet, pc}
 - Weekday = {Monday, Tuesday, ...}
- **$P(\text{Phone} \mid \text{Click}) = P(\text{Phone} \mid \text{Click \& Monday}) = P(\text{Phone} \mid \text{Click \& Tuesday})$**
- We don't assume the CTR is the same on Monday and Tuesday, but that for all clicks, the share coming from a phone is the same on Monday and Tuesday
- Given that there is a click, the value of Device is independent of the value of Weekday

$$P(X_1 = x_1, X_2 = x_2, \dots, X_k = x_k \mid Y = \text{Click}) = P(X_1 = x_1 \mid Y = \text{Click}) \times P(X_2 = x_2 \mid Y = \text{Click}) \times \dots = \prod_{i=1}^k P(X_i = x_i \mid Y = \text{Click})$$



Example

$Y = \{\text{Click, No Click}\}$

Features:

- $X_1 = \{\text{Android, iPhone}\}$
- $X_2 = \{\text{Red ad text, Blue ad text}\}$

views		
	red	blue
Android	4200	2800
iPhone	1800	1200

10,000 views

clicks		
	red	blue
Android	42	14
iPhone	12	4

72 clicks

We observe $P(\text{Click} \mid \text{iPhone, blue}) = 4 / 1200 = 0.0033$

But not very reliable based on only 1200 observations and 4 clicks

What does Naïve Bayes do?



Example

Naïve Bayes uses all observations it has for iPhones and all observations it has for blue text (independently)

views		
	red	blue
Android	4200	2800
iPhone	1800	1200

10,000 views

clicks		
	red	blue
Android	42	14
iPhone	12	4

72 clicks

	Click	No Click
$P(Y = y)$	72/10000	9928/10000
$P(X_1 = \text{Android} \mid Y = y)$	56/72	6944/9928
$P(X_1 = \text{iPhone} \mid Y = y)$	16/72	2984/9928
$P(X_2 = \text{red} \mid Y = y)$	54/72	5946/9928
$P(X_2 = \text{blue} \mid Y = y)$	18/72	3982/9928

$$P(Y = \text{Click} \mid X_1 = x_1, X_2 = x_2, \dots, X_k = x_k) =$$

$$\frac{P(Y = \text{Click}) \prod_{i=1}^k P(X_i = x_i \mid Y = \text{Click})}{\sum_y P(Y = y) \prod_{i=1}^k P(X_i = x_i \mid Y = y)}$$

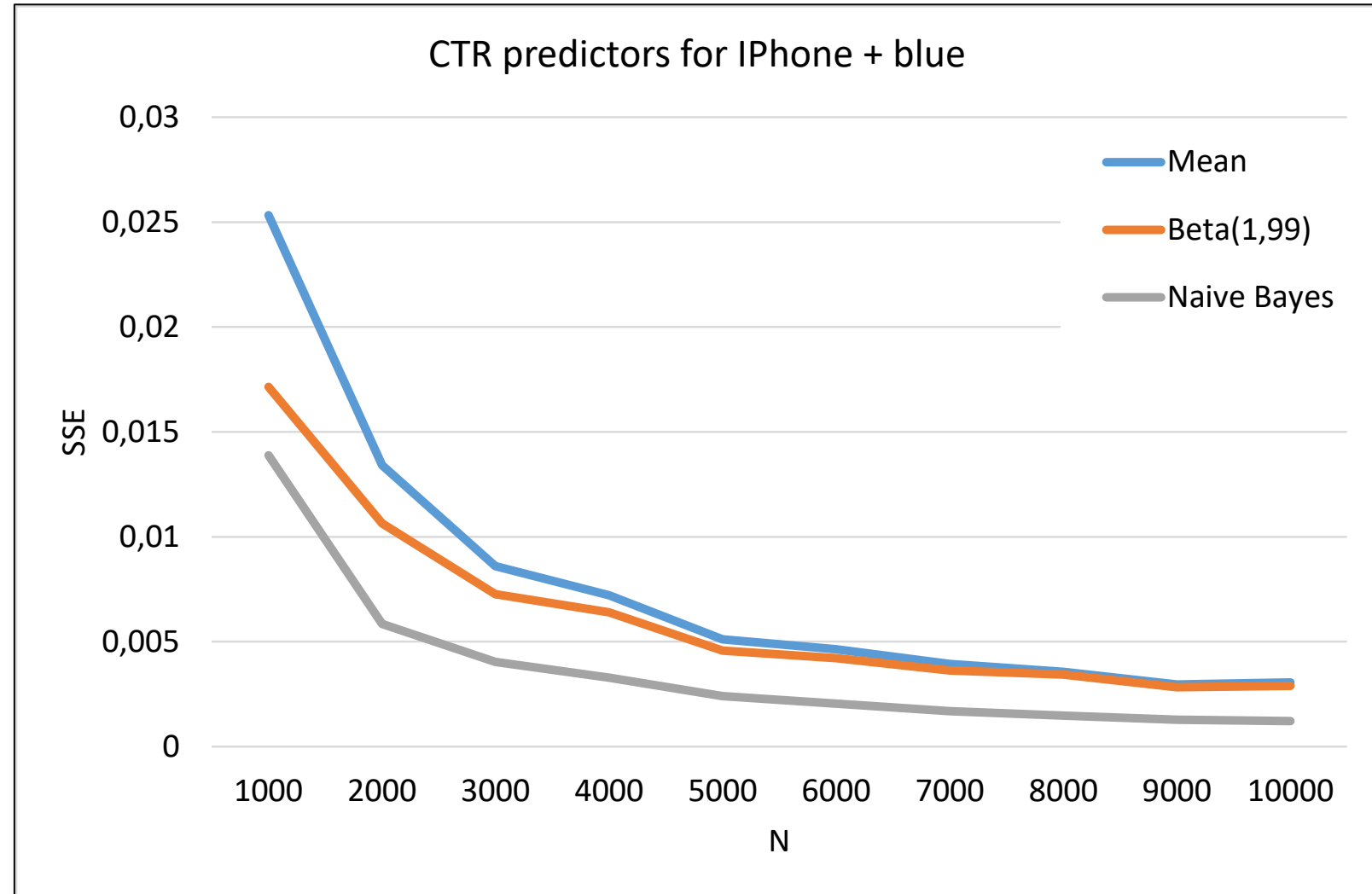
$$\begin{aligned}
 P(Y = \text{Click} \mid X_1 = \text{iPhone}, X_2 = \text{blue}) &= \frac{P(Y = \text{Click})P(X_1 = \text{iPhone} \mid Y = \text{Click})P(X_2 = \text{blue} \mid Y = \text{Click})}{\sum_y P(Y = y) P(X_1 = \text{iPhone} \mid Y = y)P(X_2 = \text{blue} \mid Y = y)} \\
 &= \frac{\frac{72}{10000} \cdot \frac{16}{72} \cdot \frac{18}{72}}{\frac{72}{10000} \cdot \frac{16}{72} \cdot \frac{18}{72} + \frac{9928}{10000} \cdot \frac{2984}{9928} \cdot \frac{3982}{9928}} = 0.0033
 \end{aligned}$$

Example

- Simulation shows that Naïve Bayes outperforms sample mean and Bayesian updating (when conditional independence holds)
- Especially when the number of observations is small

Why?

- Sample mean and Bayesian updating only use 120 of every 1000 observations. Naïve Bayes uses all observations for iPhone *and* all observations for blue





About Naive Bayes

Pros

- Fast to train (online learning)
- Very scalable
- Not sensitive to irrelevant features
- Widely used and good results in many different applications

Cons

- Assumes conditional independence of the features
- Manual definition of feature interactions and feature selection required

Note: Naïve bayes is not necessarily a Bayesian method (but can be if the parameters are estimated by Bayesian updating)



UNIVERSITY OF AMSTERDAM
Amsterdam Business School

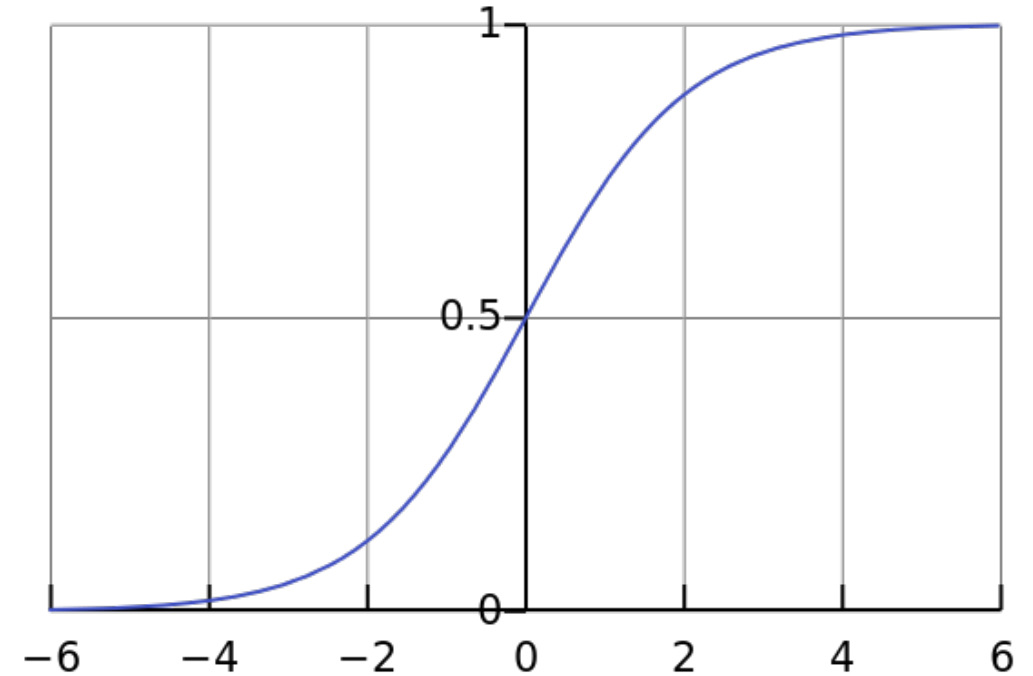
Logistic Regression



Logistic Function

$$f(x) = \frac{1}{1+e^{-x}}$$

- Transforms any value x into a value between 0 and 1
- Typically used for modeling probabilities (classification)





Logistic Regression

$$Y = \frac{1}{1 + e^{-(\beta_1 x_1 + \beta_2 x_2 + \dots)}}$$

with

- Outcome: $Y = 0$ or 1 (No Click or Click)
- Features: $X = (X_1, X_2, X_3, \dots, X_k)$
- Transform the feature values into a prediction of the probability of the outcome (value between 0-1)
- We want to find the values for the coefficients $\beta_0, \beta_1, \beta_2, \dots$ that fits our data the best
- Equivalent of linear regression for classification



Logistic Regression – Categorical features

$$Y = \frac{1}{1 + e^{-(\beta_1 x_1 + \beta_2 x_2 + \dots)}}$$

with

- Outcome: $Y = 0$ or 1 (No Click or Click)
- Features: $X = (X_1, X_2, X_3, \dots, X_k)$
- In logistic regression the feature values need to be numeric
- We can represent a categorical feature as a series of binary variables (0-1) or *dummy* variables for each of the possible values using **one hot encoding**

Logistic Regression – Categorical features

One Hot Encoding example:

- Device = {phone, tablet, pc}
- Weekday = {Monday, Tuesday, ...}



Variable	Feature	Value
X_1	Device	phone
X_2	Device	tablet
X_3	Device	pc
X_4	Weekday	Monday
X_5	Weekday	Tuesday
X_6	Weekday	Wednesday
..	..	..

- Now an observation for a pc on Tuesday is represented by the following variable values:

$$x_1 = 0, x_2 = 0, x_3 = 1, \\ x_4 = 0, x_5 = 1, x_6 = 0, ..$$

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder()

# Fit the encoder to the data in X_train and transform the data
X_train_encoded = encoder.fit_transform(X_train)

# Use the encoder to transform the data in X_test
X_test_encoded = encoder.transform(X_test)
```



Example

- $Y = 0$ or 1 (No Click or Click)

Features:

- Device: {Android, iPhone}
- Ad Text color: {Red, Blue}

Gives 10,000 observations that look like:

views		
	red	blue
Android	4200	2800
iPhone	1800	1200

10,000 views

clicks		
	red	blue
Android	42	14
iPhone	12	4

72 clicks

n	Click	Android	iPhone	red	blue
1	0	1	0	1	0
2	0	1	0	0	1
3	1	0	1	1	0
4	0	0	1	0	1

Y

X



Example

Result:

Variable	Coefficient
Android	0.4082
iPhone	0
red	-5.0035
blue	-5.7012

$$P(\text{Click} \mid \text{iPhone, blue}) = \frac{1}{1 + e^{-(\beta_1 x_1 + \beta_2 x_2 + \dots)}} = \frac{1}{1 + e^{-(0 - 5.7012)}} = 0.0033$$



But...

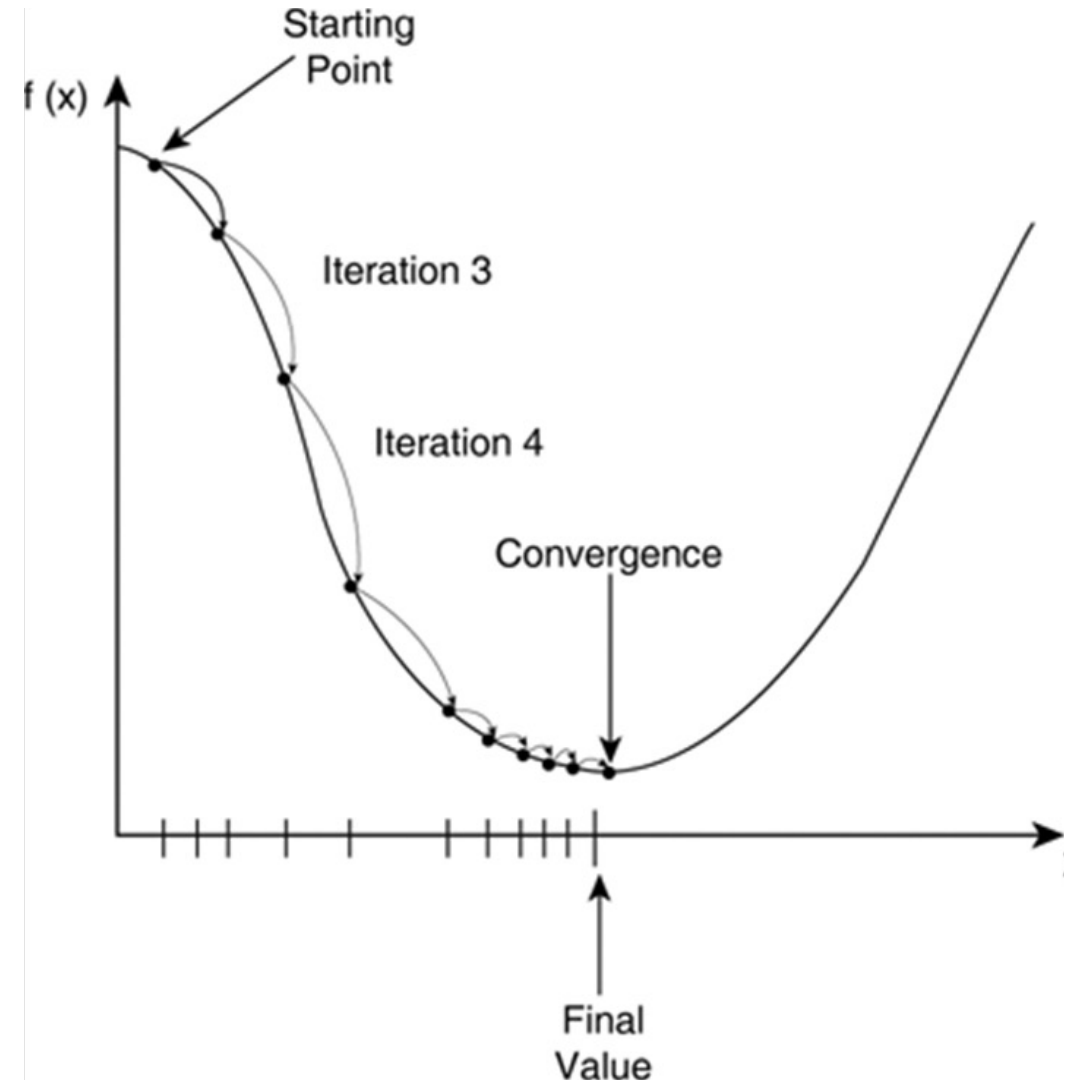
For CTR prediction, we typically have

- Many categorical features resulting in an enormous amount of dummy variables (multicollinear)
- Millions of observations

Common estimation techniques for Logistic Regression can not handle such a big matrix with multicollinearity

Stochastic Gradient Descent

- Online learning algorithm
- Can be used to fit a logistic regression model with many (categorical) features and many observations in an efficient manner!





Stochastic Gradient Descent

Minimize the **log loss** error:

$$\min \frac{1}{N} \sum_t -[y_t \ln(p_t) + (1 - y_t) \ln(1 - p_t)]$$

Where the prediction p_t is given by the logistic regression function:

$$p_t := \frac{1}{1 + e^{-(\beta_1 x_1 + \beta_2 x_2 + \dots)}}$$

We want to find the coefficients $\beta_1, \beta_2, \dots$ that minimizes:

$$\min_{\beta_1, \beta_2, \dots} \frac{1}{N} \sum_t - \left[y_t \ln \left(\frac{1}{1 + e^{-(\beta_1 x_1 + \beta_2 x_2 + \dots)}} \right) + (1 - y_t) \ln \left(1 - \frac{1}{1 + e^{-(\beta_1 x_1 + \beta_2 x_2 + \dots)}} \right) \right]$$

Stochastic Gradient Descent

Update the relevant coefficient values every time a new observation comes in:

$$\beta_i = \beta_i - \eta_t(p_t - y_t)x_{t,i}$$

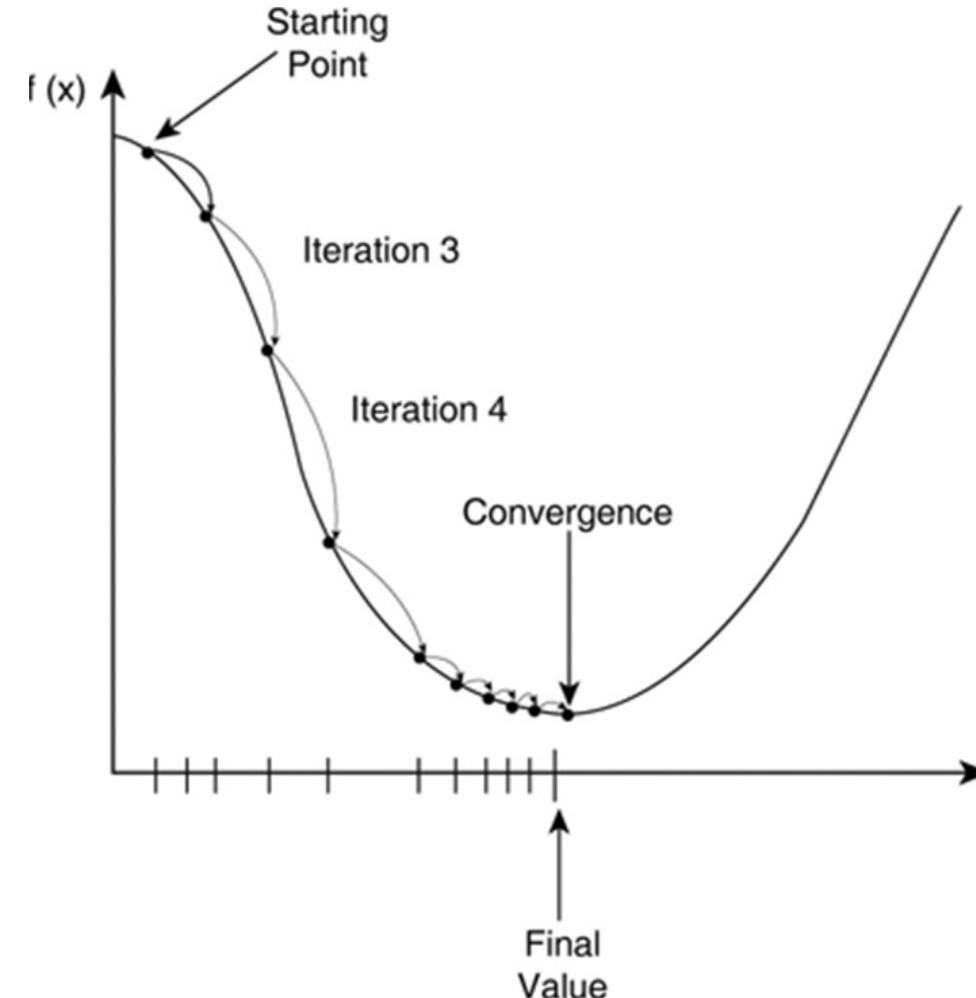
Where:

β_i = value of coefficient i

η_t = non-increasing *learning rate*, between 0-1 (e.g. $\eta_t = \frac{1}{\sqrt{t}}$)

The update is determined by the size of the error $(p_t - y_t)$ in observation t

- When we observe a click the coefficient value is increased
- When we observe a no-click the coefficient value is decreased
- No change in value for coefficients where the variable value is zero: $x_i = 0$ in observation t





Example

- $Y = 0$ or 1 (No Click or Click)

Features:

- Device: {Android, iPhone}
- Ad Text color: {Red, Blue}

Assume we have initialized our coefficients with zeroes: $\beta_{android} = \beta_{iPhone} = \beta_{red} = \beta_{blue} = 0$

Initial CTR prediction (before any observation): $p_1 = \frac{1}{1+e^{-(\beta_1 x_1 + \beta_2 x_2 + \dots)}} = \frac{1}{1+e^{-0}} = \frac{1}{2}$ for all feature values

Learning rate: $\eta_t = \frac{1}{\sqrt{t}}$, $\eta_1 = 1$

	Red	Blue
Android	0.50	0.50
iPhone	0.50	0.50



Example

Our first observation is an impression on an Android device with the red text: No Click

Example

Our first observation is an impression on an Android device with the red text: No Click

$$y_1 = 0, p_1 = \frac{1}{2}$$

Update of coefficients based on first observation:

$$\beta_{android} = \beta_{android} - \eta(p - y)x_{android} = 0 - \left(\frac{1}{2} - 0\right) \times 1 = -\frac{1}{2}$$

$$\beta_{iphone} = \beta_{iphone} - \eta(p - y)x_{iphone} = 0 - \left(\frac{1}{2} - 0\right) \times 0 = 0$$

$$\beta_{red} = \beta_{red} - \eta(p - y)x_{red} = 0 - \left(\frac{1}{2} - 0\right) \times 1 = -\frac{1}{2}$$

$$\beta_{blue} = \beta_{blue} - \eta(p - y)x_{blue} = 0 - \left(\frac{1}{2} - 0\right) \times 0 = 0$$

<i>Before</i>	Red	Blue
Android	0.50	0.50
iPhone	0.50	0.50

<i>After</i>	Red	Blue
Android	0.27	0.38
iPhone	0.38	0.50



$$p = \frac{1}{1 + e^{-(\beta_1 x_1 + \beta_2 x_2 + \dots)}}$$

Prediction for the current observation using the coefficient values from just before.

Example

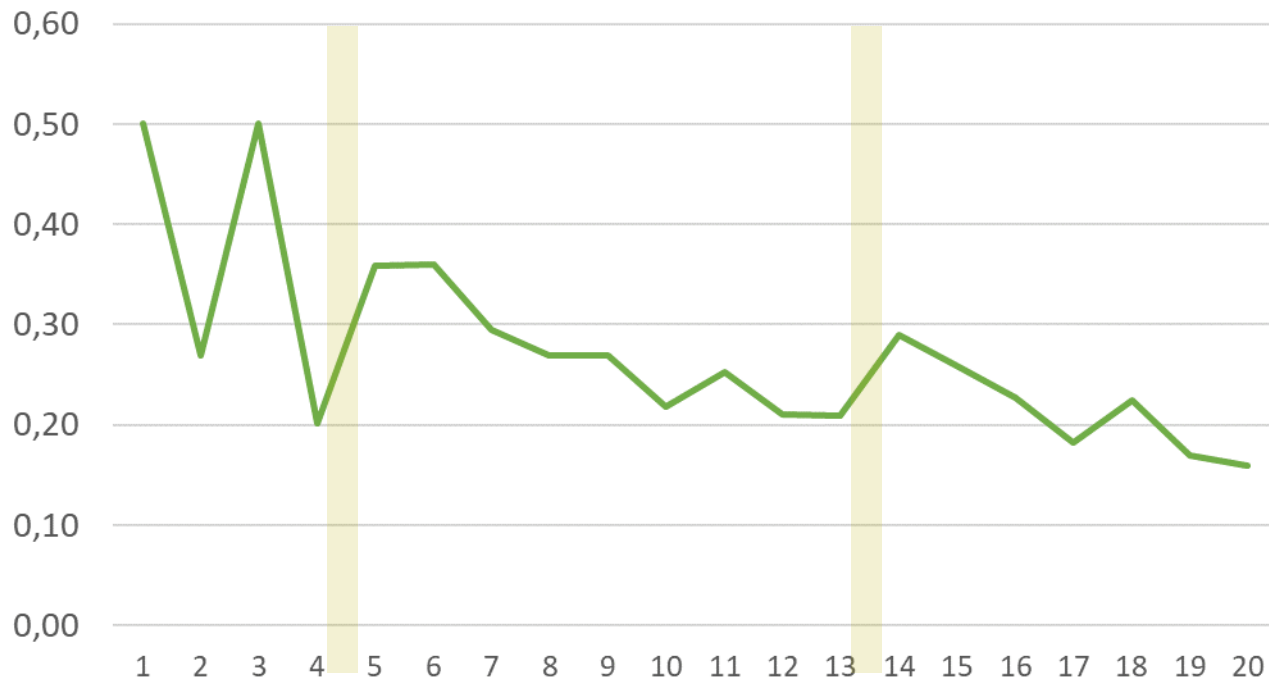
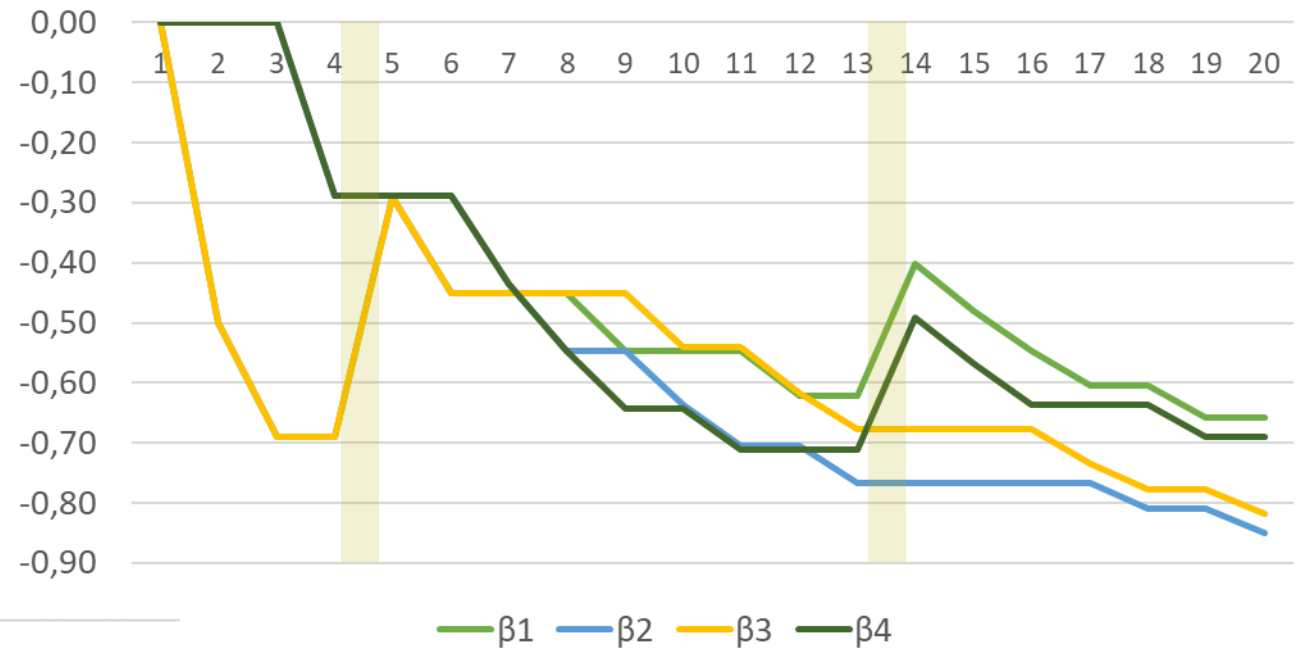
More observations..

t	$\eta = 1/\sqrt{t}$	β_1	β_2	β_3	β_4	x_1	x_2	x_3	x_4	p	Click
1	1,00	0,00	0,00	0,00	0,00	1	0	1	0	0,50	0
2	0,71	-0,50	0,00	-0,50	0,00	1	0	1	0	0,27	0
3	0,58	-0,69	0,00	-0,69	0,00	0	1	0	1	0,50	0
4	0,50	-0,69	-0,29	-0,69	-0,29	1	0	1	0	0,20	1
5	0,45	-0,29	-0,29	-0,29	-0,29	1	0	1	0	0,36	0
6	0,41	-0,45	-0,29	-0,45	-0,29	0	1	0	1	0,36	0
7	0,38	-0,45	-0,44	-0,45	-0,44	0	1	0	1	0,30	0
8	0,35	-0,45	-0,55	-0,45	-0,55	1	0	0	1	0,27	0
9	0,33	-0,55	-0,55	-0,45	-0,64	0	1	1	0	0,27	0
10	0,32	-0,55	-0,64	-0,54	-0,64	0	1	0	1	0,22	0
11	0,30	-0,55	-0,71	-0,54	-0,71	1	0	1	0	0,25	0
12	0,29	-0,62	-0,71	-0,62	-0,71	0	1	1	0	0,21	0
13	0,28	-0,62	-0,77	-0,68	-0,71	1	0	0	1	0,21	1
14	0,27	-0,40	-0,77	-0,68	-0,49	1	0	0	1	0,29	0
15	0,26	-0,48	-0,77	-0,68	-0,57	1	0	0	1	0,26	0
16	0,25	-0,55	-0,77	-0,68	-0,64	1	0	1	0	0,23	0
17	0,24	-0,60	-0,77	-0,73	-0,64	0	1	1	0	0,18	0
18	0,24	-0,60	-0,81	-0,78	-0,64	1	0	0	1	0,22	0
19	0,23	-0,66	-0,81	-0,78	-0,69	0	1	1	0	0,17	0
20	0,22	-0,66	-0,85	-0,82	-0,69	0	1	1	0	0,16	0



Example

Coefficient values



Predicted CTR



Stochastic Gradient Descent - Efficiency

- Every coefficient is updated independent of the others, based on the most recent observation only
- Not optimal, but very efficient with many categorical features

Example:

- Assume we have 10 categorical features with 100 possible values each
- Using one hot encoding this will give 1.000 (“dummy”) variables!
- But each observation only has only 10 non-zero variables
- With SGD we only need to update the values for 10 coefficients per observation



Feature Specific Learning rate

- It is common to use a specific learning rate for each coefficient separately
- This learning rate is only updated when the variable is observed ($x_i = 1$) at time t
- This way we keep learning fast for feature values that have few observations even if other features have many observations

So instead of a single learning rate η_t for observation t , we now have learning rates $\eta_{t,i}$ for each variable i

$$\eta_{t,i} = \frac{1}{\sqrt{\text{count of observations with } x_i=1}}$$



Mini-Batch SGD

- In practice you typically don't update coefficients after every new observations, but after every B new observations

Advantages

- Smoother convergence
- Computationally more efficient since vector multiplications tend to be faster than for loops



About Logistic Regression using SGD

Pros

- Fast to train (online learning)
- Very scalable

Cons

- Manual definition of feature interactions and feature selection required

Note: Naïve bayes is not necessarily a Bayesian method (but can be if the parameters are estimated by Bayesian updating)

Batch Learning

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

#Load the complete data from the file
df_data = pd.read_csv('data.csv')

model = LogisticRegression()

#Prepare data
#Split train and test set
#Split target variable (y) and features (X)
..

#train the model on all the training data at once
model.fit(X_train, y_train)
```

Online Learning

```
import pandas as pd
from sklearn.linear_model import SGDClassifier

#Prepare reading from file
chunks = pd.read_csv('data.csv', chunksize=1)

model = SGDClassifier()

#Loop over the rows of the file to read and process
for df_row in chunks:

    #Prepare data (row)
    #Split train and test?
    #Split target variable (y) and features (X)
    ..

    #Update the trained model with current row of data
    model.partial_fit(X_row, y_row, ..)
```



Reinforcement Learning: Contextual Bandits

Combine CTR prediction with many features (context) with Multi-armed Bandits

- Online learning of CTR of an ad given the context (features).
- Policy considering the **confidence of the CTR prediction to balance exploration and exploitation.**



Feature Interactions: Factorization Machines

Apply Factorization to model feature interactions:

- Regression: $\beta_1 x_1 + \beta_2 x_2 + \dots + \beta_{1,2} x_1 x_2 + \beta_{1,3} x_1 x_3 + \dots$
- Factorization machines: $\beta_1 x_1 + \beta_2 x_2 + \dots + \langle v_1 v_2 \rangle x_1 x_2 + \langle v_1 v_3 \rangle x_1 x_3 + \dots$
 - With v_i is a vector of L latent factor scores ($L \ll N$ features)
 - Instead of having to estimate N^2 coefficients we need to estimate only $N \cdot L$ latent factor scores.

$$\beta_{1,2} = \langle v_1 v_2 \rangle = [v_{1,1} \quad v_{1,2} \quad \dots \quad v_{1,L}] \begin{bmatrix} v_{2,1} \\ v_{2,2} \\ \dots \\ v_{2,L} \end{bmatrix}$$



In Practice



**McMahan et al.
(2013)**

**Predicting CTR for
Google AdWords**

Logistic regression
using SGD variant

Feature specific
learning rates used for
confidence estimates
(reinforcement
learning)



**Graepel et al.
(2010)**

**Predicting CTR for
Bing Sponsored
Search**

Probit regression
using Bayesian online
learning



He et al. (2014)

**Predicting CTR for
Facebook ads**

Logistic regression
using SGD

Encoding of features
using Boosted trees
models



Today's Course Material

- Lecture Notes
- Exam style exercises
- References are included for background information only. They don't need to be studied for the exam



References

- Graepel, Candela, Borchert & Herbrich (2010), Web-Scale Bayesian Click-Through Rate Prediction for Sponsored Search Advertising on Microsofts Bing Search Engine, Proceedings of the 27th International Conference on Machine Learning
- He, et al. (2014), Practical Lessons from Predicting Clicks on Ads at Facebook, ADKDD
- McMahan, et al. (2013), Ad Click Prediction: A View From The Trenches, KDD